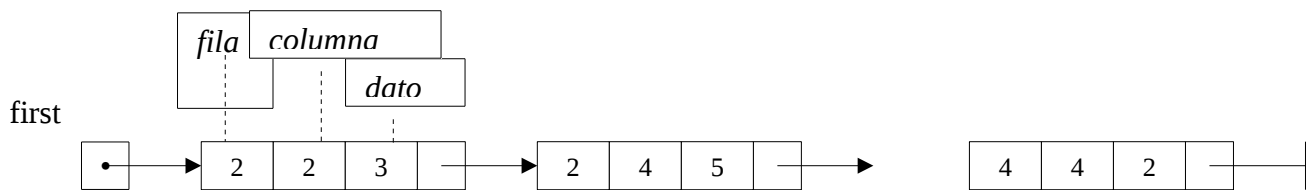


1. Suma de matrices (1,5 puntos)

Una matriz dispersa es aquella que contiene en la mayoría de sus posiciones el valor cero. Por ejemplo:

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 5 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}$$

Una matriz de este tipo se puede representar usando una lista simple con los elementos distintos de cero, ordenada ascendentemente por fila y columna:



Se pide implementar el siguiente subprograma:

```
public Node {
    Integer dato;
    Integer fila;
    Integer col;
    Node next;

    public Node(Integer pFila, Integer pCol, Integer pData){
        fila = pFila; col = pCol; dato = pData; next = null;
    }
}

public class Matriz {
    Node first;

    public Matriz suma(Matriz m1, Matriz m2) {
        // pre:
        // post: el resultado es la matriz suma de m1 y m2
    }
}
```

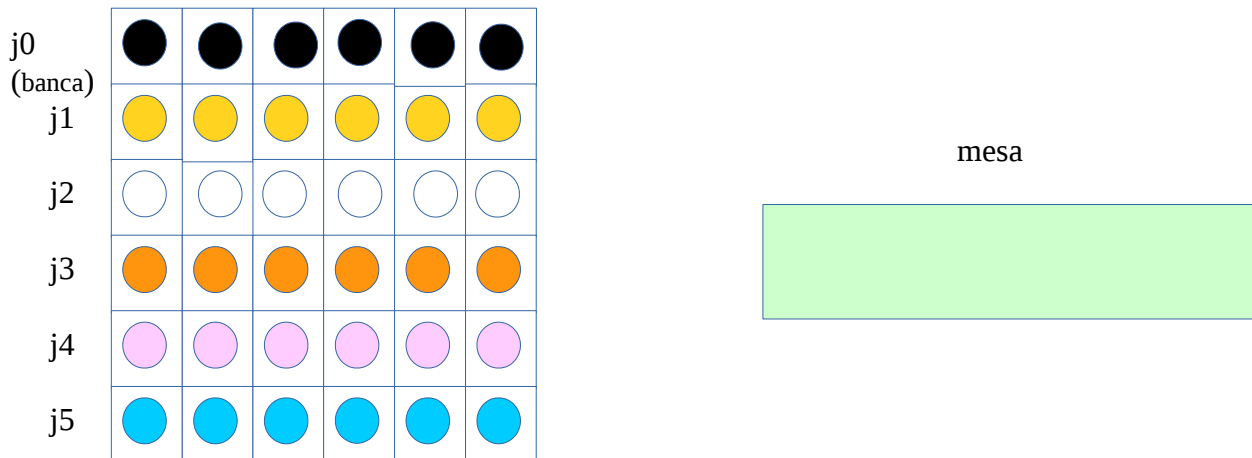
El algoritmo resultante deberá ser de coste $O(N)$, siendo N el número medio de elementos distintos de cero en $M1$ y $M2$.

Por ejemplo:

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 5 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix} + \begin{pmatrix} 9 & 0 & 0 & 0 \\ 0 & 7 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 3 \end{pmatrix} = \begin{pmatrix} 9 & 0 & 0 & 0 \\ 0 & 10 & 0 & 5 \\ 1 & 0 & 2 & 0 \\ 0 & 0 & 0 & 5 \end{pmatrix}$$

2. Juego de colores (1,5 puntos)

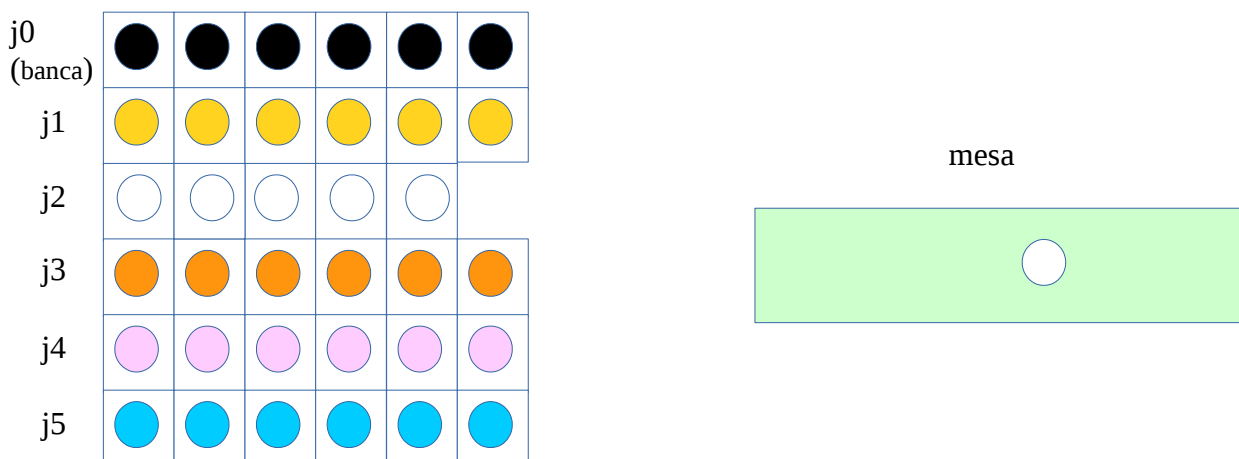
Tenemos un juego en el que un grupo de 5 jugadores tiene un número de fichas (todos los jugadores empiezan con el mismo número de fichas) de un color. Cada jugador empieza con todas sus fichas de un solo color, y todos los jugadores tienen colores diferentes. Además de los jugadores, la banca también juega con fichas de color negro (la banca será el jugador número cero). Por otra parte, el juego precisa de una mesa donde se colocan las fichas según las reglas que se indican a continuación. La siguiente figura muestra la disposición inicial del juego:



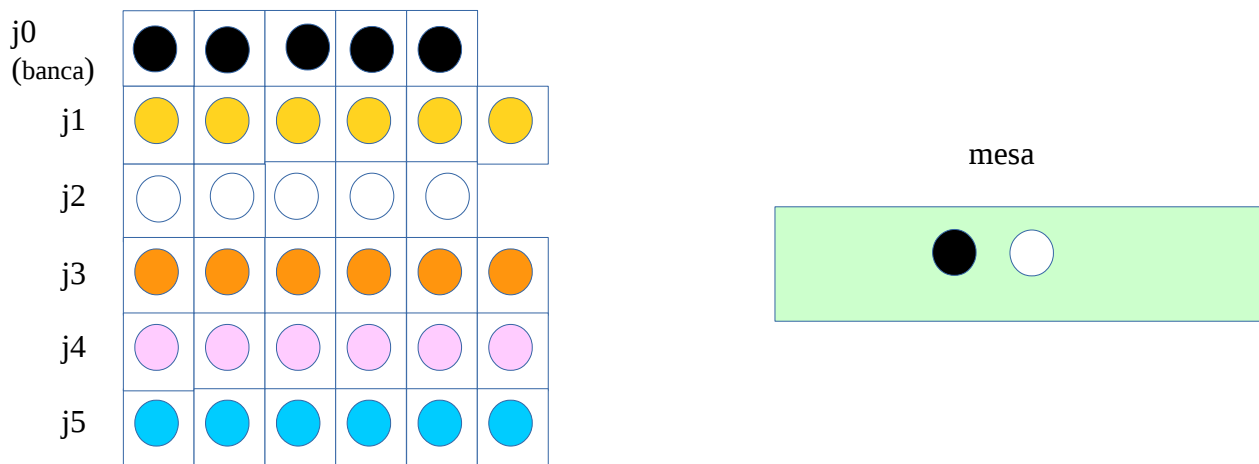
Se juega mediante las siguientes reglas, tirando dos dados en cada jugada:

1. Si el valor del primer dado es un 6, la partida acaba.
2. Si no, si el valor del primer dado es par, el valor del segundo dado indica desde qué jugador se mueve una ficha a la mesa, si hay alguna ficha. Aunque el valor del dado varía entre 1 y 6, se restará uno a ese valor para obtener un valor en el rango $[0, 5]$ (0 será la banca y el resto los 5 jugadores). No se hará nada si el jugador no tuviera fichas.
3. Si el valor del primer dado era impar, se mueve una ficha desde la mesa hasta el jugador indicado por el segundo dado. La ficha que se coja de la mesa será la última ficha que fue colocada en la mesa. Además, la ficha añadida al jugador será la última ficha en ser jugada por ese jugador. Si en la mesa no hubiera ninguna ficha, no se realizará ningún movimiento.
4. El ganador del juego será el jugador que tenga más fichas negras (sin tener en cuenta a la banca). En caso de empate, el ganador será el primer jugador con puntuación máxima.

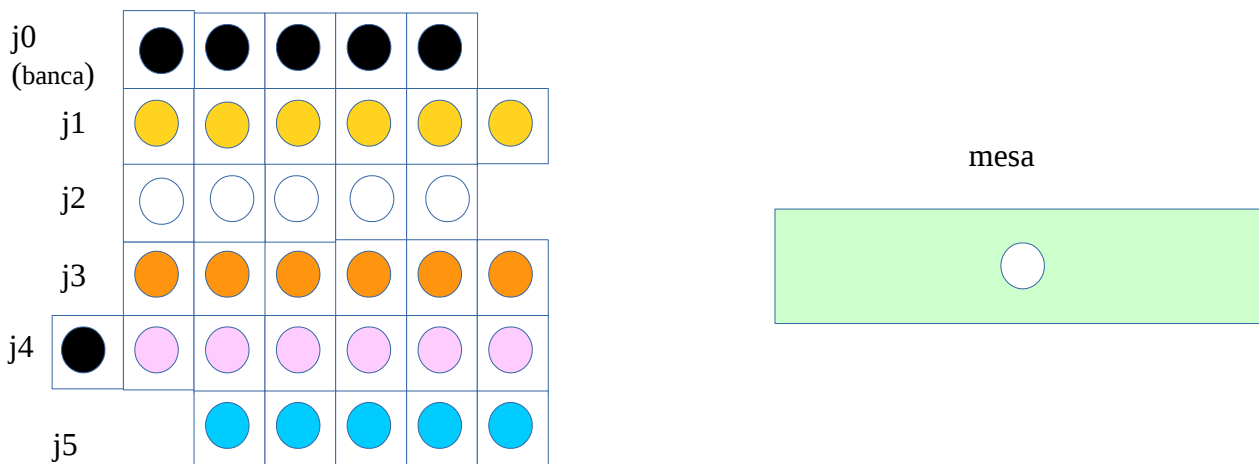
Por ejemplo, dada la situación inicial, si la primera tirada de dados devolviera (4, 3), se cogería una ficha del jugador 2 (3 menos 1) y se colocaría en la mesa:



Si ahora al tirar los dados saliera el par (4, 1) se movería una ficha desde el jugador 0 a la mesa:



Si ahora saliera el par (3, 5), al ser el primer dado impar, se movería una ficha desde la mesa al jugador 4. Se debe tener en cuenta que la ficha que se mueve debe ser la última que se colocó en la mesa. Además, esa ficha será la última en jugar por el jugador 4.



Se pide implementar el siguiente subprograma:

```
public class Juego {

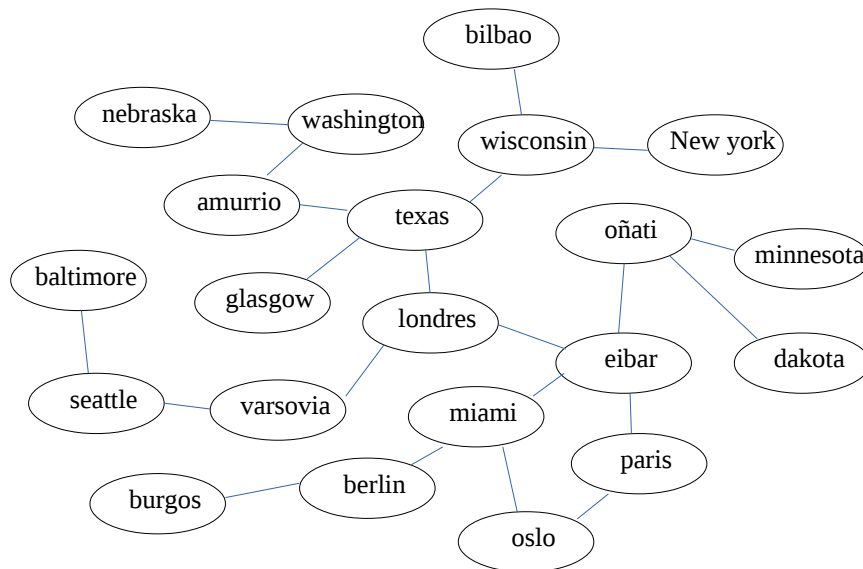
    Queue<Integer>[] jugadores;
    // Los colores de las fichas se representan por enteros ,donde las fichas
    // negras vienen dadas por el 0, y el resto de jugadores tendrán el color
    // correspondiente a la posición del jugador (es decir, el jugador 1 tendrá
    // fichas de valor 1, ...)
    Stack<Integer> mesa;

    public int juego(int n, ArrayList<Tirada> tiradas) {
        // pre: n es el número de fichas inicial de cada jugador
        //      "tiradas" tiene los valores de los dados durante una partida
        // post: el resultado es el número del jugador ganador
    }

    public class Tirada {
        int dado1;
        int dado2;
    }
}
```

3. Terremoto (1,5 puntos)

Tenemos el siguiente grafo no dirigido, que representa comarcas. Un arco entre dos comarcas indica que esas dos comarcas tienen frontera en común, es decir, son adyacentes:



Cuando se produce un terremoto de escala N en una comarca, ese terremoto se propaga a las comarcas colindantes con una intensidad igual a la mitad de N , y así sucesivamente va perdiendo fuerza.

Se quiere conocer las comarcas afectadas por un terremoto de intensidad N . Se considera que una comarca es afectada por el terremoto si la intensidad en esa comarca es igual o superior a 1.

Por ejemplo, si se produjera un terremoto de intensidad 6 en *Londres*, afectaría a *texas*, *eibar* y *varsovia* con intensidad 3, y a *glasgow*, *amurrio*, *wisconsin*, *oñati*, *paris*, *miami*, y *seattle* con intensidad 1,5.

```
public class Graph
{
    protected final int DEFAULT_CAPACITY = 100;
    protected int numVertices; // number of vertices in the graph
    protected boolean[][] adjMatrix; // adjacency matrix
    protected String[] vertices; // values of vertices

    public int index(String t) { // ¡NO HAY QUE IMPLEMENTARLO!
        // pre: el elemento t se encuentra en el grafo
        // post devuelve el índice del array "vertices" correspondiente a t
    }

    public ArrayList<String> comarcasAfectadas(int intensidad, String c)
    // pre: "intensidad" indica la magnitud del terremoto
    // "c" es la comarca en que se ha producido el terremoto
    // post: el resultado es la lista de las comarcas afectadas por el terremoto
    // de esa intensidad en la comarca dada.
    // Una comarca estará afectada si el terremoto llega con intensidad igual
    // o superior a 1.
    // La intensidad del terremoto disminuye a la mitad al pasar de una comarca
    // a las comarcas limítrofes
}
```

Se pide:

1. Implementar el método "`comarcasAfectadas()`"
2. Calcular, de manera razonada, el coste del algoritmo resultante.

4. Árbol de juego (1,5 puntos)

Tenemos un árbol que representa un juego. Los jugadores se estructuran en forma de árbol binario, y cada jugador tiene una serie de puntos asignados.

```
public class Info {
    String    s;
    Integer   puntos;
}

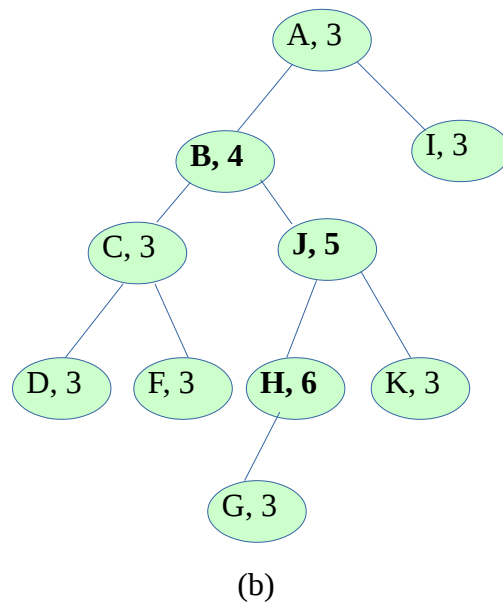
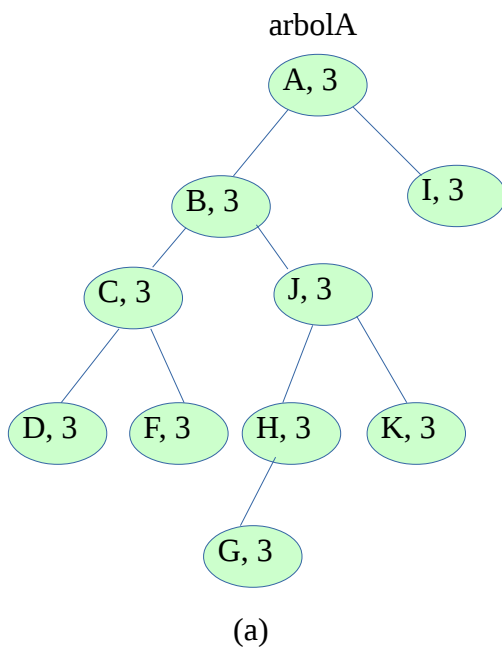
public class Nodo {
    Info      content;
    Nodo      izq, der;
    Nodo      padre;
}

public class Arbol {
    private Nodo root;

    public void premiar(int puntos, String elem);
}
```

Cuando un jugador consigue puntuar, el juego establece que se sumarán n puntos a ese jugador, y además se irán repartiendo puntos entre los antecesores de ese jugador, de manera que a su antecesor directo se le asignarán $(n-1)$ puntos, $(n-2)$ a su antecesor, y así sucesivamente.

Por ejemplo, dado el árbol de la figura (a), la llamada a “`arbolaA.premiar(3, “H”)`” produciría el árbol de la figura (b), sumando 3 puntos al jugador “H”, 2 puntos a “J” y un punto a “B”.



Se pide:

1. Implementar el método “`premiar()`”
2. Calcular, de manera razonada, el coste del algoritmo resultante.